La lógica general es:

INT xxh = qué familia de servicio uso
AH = qué función dentro de esa interrupción
otros registros = parámetros o resultados

No todo va en `AH`: `AH` selecciona la función; los datos pueden ir en `AL`, `DL`, `DX`, `BX`, `CX`, `DS:DX`, etc.

1. INT 10h — Video

Importantes

AH = 02h — Posicionar cursor

```
mov ah, 02h
mov bh, 00h      ; página
mov dh, 10       ; fila
mov dl, 20       ; columna
int 10h
```

Machete:

DH = fila
DL = columna
BH = página

AH = 09h — Mostrar carácter con atributo

Muestra un carácter varias veces, con color o atributo, pero **no avanza el cursor**.

```
mov ah, 09h
mov al, 'A'
mov bh, 00h
mov bl, 07h      ; atributo/color
mov cx, 5        ; cantidad
int 10h
```

AH = 0Ah — Mostrar carácter sin atributo nuevo

Es parecida a `09h`, pero no recibe el atributo en `BL`.

```
mov ah, 0Ah
mov al, 'A'
```

mov bh, 00h
mov cx, 5
int 10h

Tampoco avanza el cursor.

AH = 0Eh — Mostrar carácter y avanzar cursor

Es la más cómoda para mostrar caracteres uno detrás de otro.

mov ah, 0Eh
mov al, 'A'
int 10h

Machete:

> `09h`: carácter con atributo, no avanza.
> `0Eh`: carácter y avanza.

AH = 13h — Mostrar una cadena

La cadena se indica mediante `ES:BP`, la longitud mediante `CX` y la posición mediante `DH` y `DL`.

ES:BP = dirección de la cadena
CX = longitud
DH = fila
DL = columna
BL = atributo

Acá la longitud está en `CX`; **no depende de un `$` final**.

De relleno: solo reconocerlas

00h → modo de video
01h → tamaño del cursor
03h → obtener posición del cursor
05h → página activa
06h → desplazar ventana hacia arriba
07h → desplazar ventana hacia abajo
08h → leer carácter de pantalla
0Ch → escribir píxel
0Dh → leer píxel
0Fh → obtener modo de video

10h–15h → paletas, fuentes, EGA/LCD y adaptadores

La cartilla diferencia especialmente `09h`, `0Ah`, `0Eh` y `13h`, que son las más fáciles de confundir.

---

2. INT 16h — Teclado BIOS

Importantes

AH = 00h — Leer tecla

Espera hasta que haya una tecla.

mov ah, 00h
int 16h

Retorna:

AL = código ASCII
AH = código de barrido o scan code

Para teclas especiales, `AL` puede ser `00h` y el código importante queda en `AH`.

AH = 01h — Consultar si hay tecla

No espera: solamente consulta.

mov ah, 01h
int 16h
ZF = 0 → hay tecla
ZF = 1 → no hay tecla

Machete:

> `00h` lee y puede bloquear.
> `01h` pregunta sin bloquear.

AH = 02h — Estado de Shift

Devuelve en `AL` el estado de Shift, Ctrl, Alt, Caps Lock, Num Lock, etc.

De relleno

03h → velocidad y repetición
04h → click del teclado
05h → insertar carácter en el buffer
10h → lectura de teclado extendido
11h → estado del teclado extendido
12h → Shift extendido

---

3. INT 17h — Impresora

Importantes

AH = 00h — Imprimir carácter

```
mov ah, 00h
mov al, 'A'
mov dx, 0        ; LPT1
int 17h
```

DX = 0 → LPT1
DX = 1 → LPT2
AH retorna el estado

AH = 01h — Inicializar impresora

```
mov ah, 01h
mov dx, 0
int 17h
```

AH = 02h — Obtener estado

```
mov ah, 02h
mov dx, 0
int 17h
```

Retorna en `AH` si está ocupada, sin papel, seleccionada, con error o timeout.

---

4. INT 21h — Funciones de DOS

Esta es la más importante.

AH = 01h — Leer carácter con eco

Lee una tecla y la muestra en pantalla.

mov ah, 01h
int 21h

Resultado:

AL = carácter leído

AH = 02h — Mostrar un carácter

El carácter va en `DL`.

mov ah, 02h
mov dl, 'A'
int 21h

Machete:

> `02h` → un carácter → `DL`.

AH = 06h — Entrada o salida directa

Entrada

mov ah, 06h
mov dl, 0FFh
int 21h
ZF = 0 → había carácter
ZF = 1 → no había
AL = carácter

Salida

mov ah, 06h
mov dl, 'A'
int 21h

AH = 07h — Leer sin eco

mov ah, 07h
int 21h

Devuelve el carácter en `AL`, pero no lo muestra.

AH = 08h — Leer sin eco y controlar Ctrl-Break

Es similar a `07h`, pero verifica Ctrl-Break.

AH = 09h — Mostrar cadena

La dirección de la cadena va en `DS:DX` y la cadena debe terminar en `$`.

cadena db 'Hola mundo$'

mov ax, @data
mov ds, ax

mov ah, 09h
lea dx, cadena
int 21h

Errores típicos:

usar DL en lugar de DX
olvidar el $
no cargar DS
usar AH=02h para una cadena

Machete:

> `09h` → cadena → `DS:DX` → termina en `$`.

AH = 0Ah — Leer una cadena

buffer db 20
    db 0
    db 20 dup(?)

mov ah, 0Ah
lea dx, buffer
int 21h

Estructura:

byte 0 → capacidad máxima

byte 1 → cantidad leída
byte 2 en adelante → caracteres

Machete:

`09h` muestra una cadena.
`0Ah` lee una cadena.

AH = 0Bh — Consultar si hay entrada

mov ah, 0Bh
int 21h
AL = FFh → hay carácter
AL = 00h → no hay

AH = 2Ah — Fecha

mov ah, 2Ah
int 21h
CX = año
DH = mes
DL = día
AL = día de semana

AH = 2Ch — Hora

mov ah, 2Ch
int 21h
CH = hora
CL = minutos
DH = segundos
DL = centésimas

AH = 3Ch a 42h — Archivos

Machete:

3Ch → crear
3Dh → abrir
3Eh → cerrar
3Fh → leer
40h → escribir

41h → borrar
42h → mover puntero/seek

Leer archivo

mov ah, 3Fh
mov bx, HANDLE
mov cx, CANTIDAD
lea dx, buffer
int 21h

Escribir archivo

mov ah, 40h
mov bx, HANDLE
mov cx, CANTIDAD
lea dx, buffer
int 21h

Regla:

BX = handle
CX = cantidad de bytes
DS:DX = buffer

AH = 4Ch — Terminar proceso

mov ah, 4Ch
mov al, 00h
int 21h

También:

mov ax, 4C00h
int 21h

De relleno de INT 21h

Solo reconocelas:

00h → terminar programa, antigua
03h/04h → entrada/salida auxiliar
05h → impresora
0Ch → limpiar buffer y llamar otra función
0Dh–24h → discos y FCB
25h/35h → colocar/obtener vector de interrupción
2Bh/2Dh → establecer fecha/hora
30h → versión de DOS

36h → espacio libre
39h → crear directorio
3Ah → borrar directorio
3Bh → cambiar directorio
43h → atributos
48h/49h/4Ah → memoria
4Bh → cargar/ejecutar programa
4Dh → código de retorno
4Eh/4Fh → buscar archivos
56h → renombrar

Comparaciones que pueden tomarte

Mostrar carácter

INT 10h / AH=0Eh
→ carácter en AL
→ avanza cursor

INT 21h / AH=02h
→ carácter en DL

Mostrar cadena

INT 10h / AH=13h
→ ES:BP
→ CX longitud
→ DH/DL posición

INT 21h / AH=09h
→ DS:DX
→ termina en $

Leer teclado

INT 16h / 00h → BIOS, espera, ASCII + scan code
INT 16h / 01h → consulta sin esperar

INT 21h / 01h → lee con eco
INT 21h / 07h → sin eco
INT 21h / 08h → sin eco + Ctrl-Break
INT 21h / 0Ah → cadena completa

Machete UART y COM

Esta parte no está desarrollada en la cartilla de interrupciones que subiste, así que esto es un **complemento de los ejercicios que veníamos trabajando**.

Puertos base

COM1 → 3F8h
COM2 → 2F8h

Para COM2:

2F8h → registro de datos / divisor bajo
2F9h → habilitación de interrupciones / divisor alto
2FBh → Line Control Register, LCR
2FDh → Line Status Register, LSR

LCR — configuración de línea

El registro `LCR` define:

- cantidad de bits de datos;
- bits de parada;
- paridad;
- acceso al divisor mediante `DLAB`.

Ejemplo que veníamos usando:

LCR = 1Bh

Representa:

8 bits de datos
1 bit de parada
paridad par
sin break

Para acceder al divisor se activa `DLAB`, que es el bit 7 del LCR.

Velocidad

La velocidad se configura con un divisor.

Para 300 bps:

divisor = 0180h
DLL = 80h

DLM = 01h

En COM2:

```
mov dx, 2FBh
mov al, 9Bh       ; LCR con DLAB=1
out dx, al

mov dx, 2F8h
mov al, 80h       ; divisor bajo
out dx, al

mov dx, 2F9h
mov al, 01h       ; divisor alto
out dx, al
```

Luego se desactiva `DLAB` escribiendo nuevamente la configuración normal en `2FBh`.

LSR — estado del puerto

Máscaras importantes:

01h → dato disponible
02h → error de sobrecarga
04h → error de paridad
08h → error de trama
10h → break detectado
20h → registro transmisor vacío/listo

Ejemplo: verificar si hay un dato:

```
mov dx, 2FDh
in  al, dx
test al, 01h
jnz hay_dato
```

Ejemplo: verificar si se puede transmitir:

```
mov dx, 2FDh
in  al, dx
test al, 20h
jz esperar
```

Regla:

TEST máscara
JNZ → el bit está en 1
JZ → el bit está en 0

Resumen UART de memoria

COM1 = 3F8h
COM2 = 2F8h

base+0 = datos / divisor bajo
base+1 = interrupciones / divisor alto
base+3 = LCR
base+5 = LSR

LSR 01h = hay dato
LSR 20h = listo para transmitir

Machete final, el que leería antes de entrar

INT 10h
02 cursor
09 carácter con atributo, no avanza
0E carácter y avanza
13 cadena

INT 16h
00 leer
01 consultar

INT 17h
00 imprimir
01 inicializar
02 estado

INT 21h
01 leer con eco
02 mostrar carácter en DL
07 leer sin eco
08 sin eco + Ctrl-Break
09 mostrar cadena DS:DX terminada en $
0A leer cadena a buffer
0B consultar entrada
2A fecha
2C hora
3C crear
3D abrir
3E cerrar
3F leer
40 escribir

42 seek
4C terminar

COM1 3F8h
COM2 2F8h
LCR base+3
LSR base+5
LSR 01h hay dato
LSR 20h transmisor listo